

TRƯỜNG ĐẠI HỌC SƯ PHẠM HUẾ
KHOA TOÁN

— ◇ —

SỬ DỤNG SYMPY TỰ ĐỘNG KIỂM
TRA VỊ TRÍ TƯƠNG ĐỐI GIỮA ĐƯỜNG
THẲNG VÀ ĐƯỜNG TRÒN

Giảng viên giảng dạy : Nguyễn Đăng Minh Phúc

Sinh viên thực hiện : Trần Ngọc Vân Nam

Thái Thị Mỹ Hoài

Nguyễn Thị Quỳnh Như

Hà Mỹ Duyên

Huế, tháng 9 năm 2026

TRƯỜNG ĐẠI HỌC SƯ PHẠM HUẾ
KHOA TOÁN

SỬ DỤNG SYMPY TỰ ĐỘNG KIỂM
TRA VỊ TRÍ TƯƠNG ĐỐI GIỮA ĐƯỜNG
THẺNG VÀ ĐƯỜNG TRÒN

Giảng viên giảng dạy : Nguyễn Đăng Minh Phúc

Sinh viên thực hiện : Trần Ngọc Vân Nam
Thái Thị Mỹ Hoài
Nguyễn Thị Quỳnh Như
Hà Mỹ Duyên

Huế, tháng 9 năm 2026

LỜI MỞ ĐẦU

Trong quá trình học tập môn học, việc kết hợp giữa kiến thức toán học và công nghệ thông tin giúp sinh viên có cơ hội vận dụng những kiến thức lý thuyết vào các bài toán thực tế. Đặc biệt, với sự phát triển của các ngôn ngữ lập trình và thư viện hỗ trợ tính toán, nhiều bài toán toán học có thể được xử lý nhanh chóng, chính xác và tự động bằng máy tính.

Vị trí tương đối giữa đường thẳng và đường tròn là một nội dung quan trọng trong hình học giải tích. Để xác định hai đối tượng có cắt nhau, tiếp xúc hay không có điểm chung, người học thường phải thực hiện nhiều phép tính như xác định tâm, bán kính, tính khoảng cách từ tâm đến đường thẳng và so sánh các đại lượng. Việc tự động hóa những bước tính toán này giúp giảm thời gian thực hiện và hạn chế những sai sót trong quá trình tính toán.

Xuất phát từ mong muốn vận dụng kiến thức toán học vào lập trình, nhóm chúng em lựa chọn thực hiện đề tài **“Sử dụng SymPy tự động kiểm tra vị trí tương đối giữa đường thẳng và đường tròn”**. Trong đề tài, nhóm tìm hiểu cơ sở lý thuyết về đường thẳng, đường tròn và vị trí tương đối giữa hai đối tượng; đồng thời sử dụng ngôn ngữ Python kết hợp với thư viện SymPy để xây dựng chương trình tự động thực hiện các phép tính và đưa ra kết quả.

Thông qua quá trình thực hiện, nhóm có cơ hội củng cố kiến thức về hình học giải tích, tìm hiểu cách sử dụng thư viện SymPy và rèn luyện kỹ năng phân tích bài toán, xây dựng thuật toán, lập trình và kiểm thử chương trình. Bên cạnh đó, việc thực hiện bài tập theo nhóm cũng giúp các thành viên rèn luyện khả năng phối hợp, trao đổi ý tưởng và phân công công việc trong quá trình hoàn thành dự án.

Mặc dù nhóm đã cố gắng hoàn thiện đề tài, do thời gian và kiến thức còn hạn chế, bài làm khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được những ý kiến đóng góp và nhận xét từ giảng viên để có thể hoàn thiện sản phẩm cũng như nâng cao kiến thức và kỹ năng của mình trong những bài tập và dự án tiếp theo.

Nhóm chúng em xin chân thành cảm ơn!

MỤC LỤC

Mục lục

1	TỔNG QUAN VỀ ĐỀ TÀI	4
1.1	Lý do chọn đề tài	4
1.2	Mục tiêu nghiên cứu	4
1.3	Đối tượng nghiên cứu	5
1.4	Phương pháp nghiên cứu	5
2	CƠ SỞ LÝ THUYẾT	6
2.1	Phương trình đường thẳng	6
2.2	Phương trình đường tròn	6
2.3	Khoảng cách từ điểm đến đường thẳng	7
2.4	Vị trí tương đối giữa đường thẳng và đường tròn	8
2.5	Tìm tọa độ giao điểm	8
2.6	Cơ sở ứng dụng thư viện SymPy	9
2.7	Giới thiệu ngôn ngữ lập trình Python	10
2.8	Giới thiệu và khởi động thư viện SymPy	11
2.9	Vai trò của Python và SymPy trong đề tài	13
3	THIẾT KẾ THUẬT TOÁN	15
3.1	Phân tích bài toán	15
3.2	Dữ liệu đầu vào và dữ liệu ra	16
3.3	Quy trình xử lý	17
3.4	Thuật toán	18
3.5	Lưu đồ thuật toán	20
3.6	Kết luận chương 3	21
4	XÂY DỰNG CHƯƠNG TRÌNH VÀ PHÂN TÍCH MÃ NGUỒN	22
4.1	Thiết lập môi trường và phân tích biểu thức người dùng	22
4.2	Tự động trích xuất tham số và kiểm tra tính hợp lệ của đường tròn	22
4.3	Xử lý và chuyển đổi phương trình đường thẳng thành đối tượng Line	23
4.4	Tính khoảng cách và kiểm tra vị trí tương đối	24
4.5	Cơ chế kiểm soát ngoại lệ (<code>try-except</code>)	25
5	THỰC NGHIỆM VÀ ĐÁNH GIÁ	26
5.1	Các trường hợp thử nghiệm	26
5.2	Kết quả	27

5.3	Đánh giá chương trình	29
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	32
6.1	Kết quả đạt được	32
6.2	Hạn chế	32
6.3	Hướng phát triển	33
	Tài liệu tham khảo	34

1 TỔNG QUAN VỀ ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong chương trình Toán học, **vị trí tương đối giữa đường thẳng và đường tròn** là một nội dung quan trọng của hình học giải tích, giúp người học củng cố kiến thức về phương trình đường thẳng, đường tròn và rèn luyện khả năng tính toán, giải quyết bài toán hình học.

Để xác định vị trí tương đối, cần xác định tâm, bán kính đường tròn, tính khoảng cách từ tâm đến đường thẳng và so sánh với bán kính. Với các phương trình có hệ số lớn hoặc biểu thức phức tạp, việc tính toán thủ công có thể mất thời gian và dễ xảy ra sai sót. Vì vậy, việc ứng dụng công nghệ thông tin vào tính toán và kiểm tra kết quả là cần thiết.

SymPy, một thư viện mã nguồn mở của Python, hỗ trợ nhiều phép tính đại số như rút gọn biểu thức, giải phương trình và hệ phương trình. Việc ứng dụng SymPy vào bài toán giúp tự động hóa quá trình tính toán, giảm sai sót và kết hợp kiến thức toán học với kỹ năng lập trình.

Xuất phát từ đó, đề tài **“Sử dụng SymPy tự động kiểm tra vị trí tương đối giữa đường thẳng và đường tròn”** được thực hiện nhằm xây dựng chương trình tự động phân tích dữ liệu, thực hiện các phép tính cần thiết và đưa ra kết luận về vị trí tương đối giữa hai đối tượng. Qua đó, đề tài góp phần rèn luyện tư duy thuật toán và khả năng ứng dụng công nghệ thông tin vào giải quyết các bài toán toán học.

1.2 Mục tiêu nghiên cứu

Mục tiêu của đề tài là tìm hiểu và xây dựng một chương trình Python sử dụng thư viện **SymPy** để hỗ trợ tự động xác định vị trí tương đối giữa đường thẳng và đường tròn trong mặt phẳng tọa độ.

Đề tài tập trung vào việc vận dụng các kiến thức toán học về đường thẳng và đường tròn, kết hợp với khả năng tính toán của thư viện **SymPy** để tự động hóa quá trình xử lý và kiểm tra kết quả.

Thông qua đề tài, người thực hiện hướng đến các mục tiêu:

- Củng cố kiến thức về đường thẳng, đường tròn và vị trí tương đối trong hình học giải tích.
- Tìm hiểu cách sử dụng thư viện **SymPy** trong tính toán và xử lý các bài toán toán học.
- Xây dựng chương trình có khả năng tự động xử lý dữ liệu và đưa ra kết quả về vị trí tương đối giữa đường thẳng và đường tròn.

- Kiểm tra và đánh giá kết quả hoạt động của chương trình thông qua một số bài toán cụ thể.
- Rèn luyện tư duy thuật toán, kỹ năng lập trình Python và khả năng kết hợp kiến thức toán học với công nghệ thông tin.

1.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của đề tài là bài toán xác định vị trí tương đối giữa đường thẳng và đường tròn trong mặt phẳng tọa độ.

Đề tài tập trung nghiên cứu các nội dung chính sau:

- Các kiến thức cơ bản về đường thẳng và đường tròn trong mặt phẳng tọa độ.
- Các phương pháp xác định vị trí tương đối giữa đường thẳng và đường tròn.
- Việc ứng dụng kiến thức toán học vào quá trình xây dựng thuật toán và chương trình.
- Thư viện SymPy và khả năng hỗ trợ tính toán, xử lý các bài toán đại số.
- Ngôn ngữ lập trình Python được sử dụng để xây dựng và triển khai chương trình.

1.4 Phương pháp nghiên cứu

Đề tài kết hợp phương pháp nghiên cứu toán học với phương pháp lập trình nhằm xây dựng chương trình có cơ sở lý thuyết và khả năng thực hiện trên máy tính.

Phương pháp nghiên cứu lý thuyết:

Tìm hiểu các kiến thức toán học liên quan đến đường thẳng, đường tròn và vị trí tương đối giữa hai đối tượng. Đây là cơ sở để phân tích và giải quyết bài toán.

Phương pháp tìm hiểu công cụ:

Tìm hiểu ngôn ngữ lập trình **Python** và thư viện **SymPy**, đặc biệt là khả năng hỗ trợ tính toán và xử lý các bài toán toán học.

Phương pháp phân tích và xây dựng chương trình:

Phân tích yêu cầu của bài toán, xác định cách xử lý dữ liệu và xây dựng chương trình theo các bước phù hợp. Chương trình được thiết kế theo hướng đơn giản, rõ ràng và dễ sử dụng.

Phương pháp kiểm thử và đánh giá:

Tiến hành kiểm tra chương trình với các dữ liệu khác nhau, đối chiếu kết quả với kiến thức toán học để đánh giá độ chính xác và khả năng hoạt động của chương trình.

Thông qua các phương pháp trên, đề tài kết hợp kiến thức toán học với công nghệ thông tin, đồng thời rèn luyện tư duy thuật toán và kỹ năng lập trình Python.

2 CƠ SỞ LÝ THUYẾT

2.1 Phương trình đường thẳng

Trong mặt phẳng tọa độ Oxy , phương trình tổng quát của đường thẳng có dạng:

$$ax + by + c = 0$$

trong đó a, b, c là các số thực và $a^2 + b^2 \neq 0$.

Trong phương trình trên, vectơ

$$\vec{n} = (a, b)$$

là một vectơ pháp tuyến của đường thẳng.

Nếu $b \neq 0$, phương trình đường thẳng có thể được viết dưới dạng hàm số:

$$y = -\frac{a}{b}x - \frac{c}{b}.$$

Hệ số góc của đường thẳng là:

$$k = -\frac{a}{b}.$$

Ngoài dạng tổng quát, đường thẳng còn có thể được biểu diễn dưới dạng:

$$y = kx + m,$$

trong đó k là hệ số góc và m là tung độ giao điểm của đường thẳng với trục Oy .

Trong bài toán xác định vị trí tương đối giữa đường thẳng và đường tròn, phương trình tổng quát

$$ax + by + c = 0$$

được sử dụng để tính khoảng cách từ tâm đường tròn đến đường thẳng.

2.2 Phương trình đường tròn

Trong mặt phẳng tọa độ Oxy , đường tròn có tâm $I(x_0, y_0)$ và bán kính $R > 0$ có phương trình:

$$(x - x_0)^2 + (y - y_0)^2 = R^2.$$

Trong đó:

- $I(x_0, y_0)$ là tâm của đường tròn;

- R là bán kính của đường tròn.

Phương trình đường tròn dạng tổng quát có thể được viết dưới dạng:

$$x^2 + y^2 + Dx + Ey + F = 0.$$

Từ phương trình tổng quát, tọa độ tâm và bán kính được xác định bởi:

$$x_0 = -\frac{D}{2}, \quad y_0 = -\frac{E}{2},$$

và

$$R = \sqrt{\frac{D^2 + E^2}{4} - F}.$$

Điều kiện để phương trình trên biểu diễn một đường tròn thực là:

$$\frac{D^2 + E^2}{4} - F > 0.$$

Việc xác định chính xác tâm và bán kính của đường tròn là cơ sở quan trọng để thực hiện bài toán kiểm tra vị trí tương đối giữa đường thẳng và đường tròn.

2.3 Khoảng cách từ điểm đến đường thẳng

Cho điểm $M(x_0, y_0)$ và đường thẳng:

$$\Delta : ax + by + c = 0.$$

Khoảng cách từ điểm M đến đường thẳng Δ được tính theo công thức:

$$d(M, \Delta) = \frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}.$$

Công thức trên cho phép xác định khoảng cách ngắn nhất từ một điểm đến một đường thẳng.

Trong bài toán đang xét, điểm M chính là tâm $I(x_0, y_0)$ của đường tròn. Do đó, khoảng cách từ tâm đường tròn đến đường thẳng được tính bởi:

$$d(I, \Delta) = \frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}.$$

Sau khi tính được khoảng cách d , ta tiến hành so sánh d với bán kính R của đường tròn để xác định vị trí tương đối giữa đường thẳng và đường tròn.

2.4 Vị trí tương đối giữa đường thẳng và đường tròn

Cho đường tròn (C) có tâm $I(x_0, y_0)$, bán kính R và đường thẳng:

$$\Delta : ax + by + c = 0.$$

Gọi d là khoảng cách từ tâm I đến đường thẳng Δ :

$$d = \frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}.$$

Vị trí tương đối giữa đường thẳng và đường tròn được xác định dựa trên việc so sánh d với R .

Đường thẳng không có điểm chung với đường tròn

Nếu $d > R$ thì đường thẳng nằm ngoài đường tròn và không có điểm chung với đường tròn. Khi đó, phương trình hệ giữa đường thẳng và đường tròn không có nghiệm thực.

Đường thẳng tiếp xúc với đường tròn

Nếu $d = R$ thì đường thẳng tiếp xúc với đường tròn tại đúng một điểm. Điểm đó được gọi là tiếp điểm của đường thẳng và đường tròn.

Trong trường hợp này, hệ phương trình gồm phương trình đường thẳng và phương trình đường tròn có đúng một nghiệm thực.

Đường thẳng cắt đường tròn tại hai điểm

Nếu $d < R$ thì đường thẳng cắt đường tròn tại hai điểm phân biệt. Khi đó, hệ phương trình giữa đường thẳng và đường tròn có hai nghiệm thực phân biệt.

Bảng tổng hợp vị trí tương đối

Có thể tổng hợp các trường hợp vị trí tương đối giữa đường thẳng và đường tròn như sau:

Điều kiện	Số điểm chung	Vị trí tương đối
$d > R$	0	Không có điểm chung
$d = R$	1	Tiếp xúc
$d < R$	2	Cắt nhau

Đây là cơ sở lý thuyết chính được sử dụng để xây dựng thuật toán tự động kiểm tra vị trí tương đối giữa đường thẳng và đường tròn bằng Python và thư viện SymPy.

2.5 Tìm tọa độ giao điểm

Ngoài việc xác định số điểm chung, trong trường hợp đường thẳng cắt đường tròn hoặc tiếp xúc với đường tròn, ta có thể tìm tọa độ các điểm chung bằng cách giải hệ phương trình:

$$\begin{cases} ax + by + c = 0, \\ (x - x_0)^2 + (y - y_0)^2 = R^2. \end{cases}$$

Từ phương trình đường thẳng, nếu $b \neq 0$, ta có:

$$y = -\frac{ax + c}{b}.$$

Thay biểu thức của y vào phương trình đường tròn:

$$(x - x_0)^2 + \left(-\frac{ax + c}{b} - y_0\right)^2 = R^2.$$

Sau khi biến đổi, ta thu được một phương trình bậc hai theo x :

$$Ax^2 + Bx + C = 0.$$

Số nghiệm của phương trình này quyết định số giao điểm giữa đường thẳng và đường tròn.

Nếu phương trình có hai nghiệm phân biệt thì đường thẳng cắt đường tròn tại hai điểm.

Nếu phương trình có một nghiệm kép thì đường thẳng tiếp xúc với đường tròn.

Nếu phương trình không có nghiệm thực thì đường thẳng không có điểm chung với đường tròn.

Trong trường hợp sử dụng thư viện SymPy, việc giải hệ phương trình có thể được thực hiện tự động. SymPy cung cấp các công cụ đại số ký hiệu giúp máy tính thực hiện các phép biến đổi và tìm nghiệm của hệ phương trình một cách chính xác.

2.6 Cơ sở ứng dụng thư viện SymPy

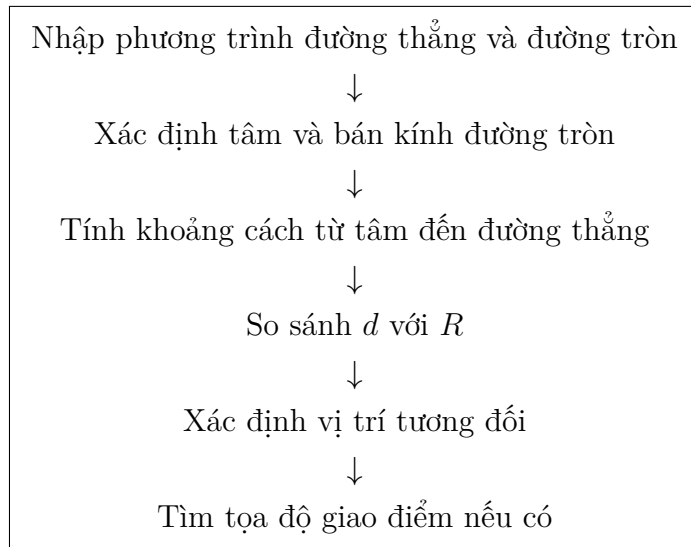
SymPy là một thư viện Python được sử dụng để thực hiện các phép tính toán học theo phương pháp đại số ký hiệu. Thư viện hỗ trợ nhiều chức năng như biến đổi biểu thức, giải phương trình, giải hệ phương trình, tính đạo hàm, tích phân và xử lý các đối tượng hình học.

Trong bài toán xác định vị trí tương đối giữa đường thẳng và đường tròn, SymPy có thể được sử dụng để:

- Khai báo các biến và tham số toán học;
- Biểu diễn phương trình đường thẳng;
- Biểu diễn phương trình đường tròn;
- Xác định tâm và bán kính của đường tròn;

- Tính khoảng cách từ tâm đường tròn đến đường thẳng;
- So sánh khoảng cách với bán kính;
- Giải hệ phương trình để tìm tọa độ giao điểm;
- Hiển thị kết quả tính toán một cách chính xác.

Dựa trên các cơ sở lý thuyết trên, chương trình có thể xây dựng quy trình xử lý như sau:



Như vậy, cơ sở lý thuyết của bài toán bao gồm các kiến thức về phương trình đường thẳng, phương trình đường tròn, khoảng cách từ điểm đến đường thẳng và vị trí tương đối giữa đường thẳng với đường tròn. Đây là nền tảng để xây dựng thuật toán và chương trình tự động kiểm tra bằng Python kết hợp với thư viện SymPy.

2.7 Giới thiệu ngôn ngữ lập trình Python

Python là một ngôn ngữ lập trình bậc cao, có cú pháp đơn giản, dễ đọc và được sử dụng rộng rãi trong nhiều lĩnh vực như phát triển phần mềm, phân tích dữ liệu, trí tuệ nhân tạo và tính toán khoa học. Với hệ thống thư viện phong phú, Python cho phép người lập trình thực hiện các phép tính toán học và xây dựng các chương trình giải quyết bài toán một cách thuận tiện.

Trong đề tài này, Python được sử dụng làm ngôn ngữ chính để xây dựng chương trình giải các bài toán liên quan đến đường thẳng và đường tròn trong mặt phẳng tọa độ. Chương trình có khả năng tiếp nhận dữ liệu đầu vào, thực hiện các phép tính cần thiết và đưa ra kết quả cho người sử dụng.

Một số ưu điểm của Python phù hợp với đề tài gồm:

- Cú pháp đơn giản, dễ học và dễ triển khai;

- Hỗ trợ tốt các phép tính toán học;
- Có nhiều thư viện phục vụ tính toán khoa học;
- Cho phép xử lý số và biểu thức đại số một cách linh hoạt;
- Dễ dàng kết hợp với các thư viện như SymPy để giải các bài toán toán học.

Do đó, Python là công cụ phù hợp để xây dựng chương trình hỗ trợ giải các bài toán hình học giải tích được trình bày trong đề tài.

2.8 Giới thiệu và khởi động thư viện SymPy

SymPy là một thư viện mã nguồn mở của ngôn ngữ lập trình Python, được sử dụng để thực hiện các phép tính toán học dưới dạng ký hiệu. Thư viện hỗ trợ nhiều chức năng như khai báo biến, biến đổi biểu thức đại số, giải phương trình, giải hệ phương trình và thực hiện các phép tính liên quan đến hình học giải tích.

Trong đề tài, SymPy được sử dụng để hỗ trợ xử lý các bài toán về đường thẳng và đường tròn, đặc biệt là tính toán khoảng cách, xác định vị trí tương đối và tìm tọa độ giao điểm.

Để khởi động môi trường Python và sử dụng SymPy trực tiếp trên cửa sổ Command Prompt (CMD), trước tiên mở CMD bằng cách nhấn tổ hợp phím: **Windows + R**

Sau đó nhập: `cmd` và nhấn phím **Enter**.

Trong cửa sổ CMD, nhập lệnh:

```
python
```

để khởi động môi trường Python. Khi Python được khởi động thành công, cửa sổ CMD sẽ xuất hiện dấu nhắc: `>>>`

Tiếp theo, sử dụng các lệnh:

```
import sys
from sympy import symbols, Eq
from sympy.parsing.sympy_parser import parse_expr
from sympy.geometry import Point, Line, Circle

# Khai tạo hai biến ký hiệu x và y
x, y = symbols('x y')
```

Các lệnh trên được sử dụng để khai báo các thư viện và thành phần cần thiết cho chương trình. Cụ thể:

- **sys**: là thư viện có sẵn của Python, hỗ trợ làm việc với hệ thống và môi trường thực thi của chương trình.

- **symbols**: được sử dụng để khai báo các biến toán học dưới dạng ký hiệu. Trong chương trình, các biến x , y có thể được khai báo để sử dụng trong các phương trình và biểu thức.
- **Eq**: được sử dụng để tạo và biểu diễn các phương trình toán học.
- **parse_expr**: được sử dụng để phân tích biểu thức được nhập dưới dạng chuỗi và chuyển thành biểu thức toán học mà SymPy có thể xử lý.
- **Point**: được sử dụng để biểu diễn một điểm trong mặt phẳng tọa độ.
- **Line**: được sử dụng để biểu diễn đường thẳng trong mặt phẳng.
- **Circle**: được sử dụng để biểu diễn đường tròn trong mặt phẳng.

Sau khi khai báo các thư viện cần thiết, chương trình có thể bắt đầu xây dựng các đối tượng và biểu thức toán học phục vụ cho bài toán.

Trước hết, các biến toán học được khai báo bằng hàm **symbols()**:

```
x, y = symbols('x y')
```

Trong đó, x và y là các biến ký hiệu được sử dụng để biểu diễn tọa độ của các điểm trong mặt phẳng. Việc sử dụng biến ký hiệu giúp chương trình có thể thực hiện các phép biến đổi và xử lý biểu thức toán học một cách linh hoạt.

Để biểu diễn một điểm trong mặt phẳng tọa độ, sử dụng lớp **Point**. Ví dụ:

```
A = Point(2, 3)
```

Câu lệnh trên tạo điểm $A(2, 3)$ trong mặt phẳng tọa độ.

Để biểu diễn đường thẳng đi qua hai điểm, sử dụng lớp **Line**. Ví dụ:

```
A = Point(0, 0)
B = Point(4, 2)
d = Line(A, B)
```

Trong đó, d là đường thẳng đi qua hai điểm A và B .

Để biểu diễn đường tròn, sử dụng lớp **Circle**. Ví dụ:

```
I = Point(1, 2)
C = Circle(I, 3)}
```

Trong đó, $I(1, 2)$ là tâm của đường tròn và bán kính bằng 3.

Bên cạnh việc xây dựng các đối tượng hình học, SymPy còn hỗ trợ biểu diễn các phương trình thông qua lớp **Eq**. Ví dụ:

```
x, y = symbols('x y')
pt = Eq(x + y, 5)
```

Câu lệnh trên biểu diễn phương trình: $x + y = 5$.

Trong chương trình, `parse_expr` được sử dụng để phân tích các biểu thức được nhập dưới dạng chuỗi. Điều này giúp chương trình có thể tiếp nhận biểu thức toán học từ người sử dụng và chuyển đổi thành dạng biểu thức mà SymPy có thể xử lý.

Ví dụ, khi người sử dụng nhập một biểu thức dưới dạng chuỗi, chương trình có thể sử dụng:

```
expr = parse_expr("x + 2*y")
```

Khi đó, biểu thức được chuyển thành biểu thức toán học: $x + 2y$.

Việc kết hợp các thành phần trên giúp chương trình có thể xử lý các bài toán hình học giải tích theo một quy trình thống nhất. Đối với bài toán về đường thẳng và đường tròn, quy trình xử lý có thể được thực hiện theo các bước sau:

1. Khởi động môi trường Python trực tiếp trên cửa sổ CMD.
2. Khai báo các thư viện và thành phần cần thiết của SymPy.
3. Khai báo các biến toán học bằng `symbols()`.
4. Tiếp nhận và phân tích dữ liệu đầu vào bằng `parse_expr`.
5. Xây dựng các đối tượng hình học bằng `Point`, `Line` và `Circle`.
6. Xây dựng các phương trình cần thiết bằng `Eq`.
7. Thực hiện các phép tính toán học và hình học dựa trên dữ liệu đầu vào.
8. Xác định kết quả của bài toán và hiển thị cho người sử dụng.

Đối với bài toán đường thẳng và đường tròn, Python và SymPy có thể được sử dụng để thực hiện các nội dung chính như xác định phương trình đường thẳng, xác định phương trình đường tròn, tính khoảng cách từ một điểm đến đường thẳng, xác định vị trí tương đối giữa đường thẳng và đường tròn, đồng thời tìm tọa độ các giao điểm.

2.9 Vai trò của Python và SymPy trong đề tài

Python đóng vai trò là ngôn ngữ lập trình chính, chịu trách nhiệm xây dựng cấu trúc chương trình, tiếp nhận dữ liệu đầu vào, xử lý dữ liệu và hiển thị kết quả. Trong khi đó, SymPy cung cấp các công cụ toán học và hình học cần thiết để chương trình có thể xử lý các biểu thức và đối tượng hình học.

Sự kết hợp giữa Python và SymPy giúp quá trình giải bài toán được thực hiện theo hướng tự động hóa. Thay vì thực hiện toàn bộ các phép tính bằng phương pháp thủ công, người sử dụng chỉ cần cung cấp dữ liệu đầu vào, chương trình sẽ tiến hành xử lý và đưa ra kết quả tương ứng.

Trong phạm vi đề tài, Python và SymPy được sử dụng để hỗ trợ giải quyết các bài toán liên quan đến đường thẳng và đường tròn trong mặt phẳng tọa độ. Các kết quả tính toán của chương trình được xây dựng dựa trên các công thức toán học đã trình bày ở các phần cơ sở lý thuyết trước đó.

Như vậy, việc sử dụng Python kết hợp với thư viện SymPy vừa đảm bảo cơ sở toán học của bài toán, vừa tạo điều kiện thuận lợi để xây dựng một chương trình có khả năng tính toán và xử lý các bài toán hình học giải tích một cách tự động.

3 THIẾT KẾ THUẬT TOÁN

Chương này trình bày toàn diện quá trình phân tích bài toán, xây dựng mô hình đại số ký hiệu và thiết kế thuật toán chi tiết cho chương trình tự động kiểm tra vị trí tương đối giữa đường thẳng và đường tròn. Thuật toán được thiết kế dựa trên sự kết hợp chặt chẽ giữa hình học giải tích phẳng và các mô-đun xử lý biểu thức ký hiệu, phân tích cú pháp biểu tượng của thư viện SymPy, nhằm đảm bảo độ chính xác cao, tránh các sai số số thực và cung cấp khả năng xử lý biểu thức linh hoạt.

3.1 Phân tích bài toán

Khác với các phương pháp tính toán số học truyền thống vốn yêu cầu người dùng phải tách rời từng hệ số thủ công, mô hình thuật toán này cho phép tiếp nhận trực tiếp biểu thức phương trình dưới dạng chuỗi ký tự (string) và chuyển hóa chúng thành cấu trúc dữ liệu ký hiệu thông qua thư viện SymPy.

- **Đường tròn (C):** Phương trình đường tròn được người dùng nhập vào dưới dạng biểu thức vế trái (sau khi đã quy đồng và chuyển vế để vế phải bằng 0), đại diện cho phương trình khai triển tổng quát của đường tròn trên hệ trục tọa độ Descartes:

$$(C) : x^2 + y^2 + 2gx + 2fy + c = 0$$

Để trích xuất các thông số hình học cốt lõi từ chuỗi ký hiệu này, thuật toán thực hiện các bước phân tích sau:

1. Sử dụng hàm `parse_expr` để phân tích cú pháp chuỗi ký tự thành biểu thức toán học ký hiệu của SymPy.
2. Trích xuất hệ số của các biến ẩn x và y thông qua phương thức `.coeff(x)` và `.coeff(y)` để xác định các thành phần $2g$ và $2f$.
3. Xác định hệ số tự do c bằng cách thay giá trị $x = 0$ và $y = 0$ vào biểu thức (`.subs({x: 0, y: 0})`).
4. Từ các giá trị trên, tọa độ tâm $I(a, b)$ và bình phương bán kính R^2 được tính toán tự động:

$$a = -\frac{\text{coeff}(x)}{2}, \quad b = -\frac{\text{coeff}(y)}{2}, \quad R^2 = a^2 + b^2 - c$$

5. Điều kiện ràng buộc hợp lệ: Giá trị bình phương bán kính phải thỏa mãn $R^2 > 0$. Nếu điều kiện này vi phạm, biểu thức không cấu thành một đường tròn thực trên mặt phẳng tọa độ, chương trình sẽ lập tức phát sinh cảnh báo và chấm dứt thực thi để đảm bảo an toàn logic.

- **Đường thẳng Δ :** Đường thẳng được biểu diễn thông qua phương trình tổng quát:

$$\Delta : Ax + By + C = 0$$

Tương tự như đường tròn, chuỗi phương trình đường thẳng được phân tích cú pháp bằng `parse_expr`. Các hệ số A, B lần lượt được trích xuất từ hệ số của x và y , trong khi hằng số C được xác định thông qua phép thế giá trị gốc tọa độ $(0, 0)$.

Điều kiện ràng buộc hợp lệ: Để phương trình xác định duy nhất một đường thẳng, tổng bình phương các hệ số góc phải thỏa mãn:

$$A^2 + B^2 > 0$$

Điều này đồng nghĩa với việc A và B không thể đồng thời bằng 0.

Bên cạnh đó, cơ sở xác định vị trí tương đối hình học được thực hiện sau khi trích xuất thành công các tham số và khởi tạo các đối tượng hình học tiêu chuẩn của SymPy bao gồm `Circle` (đường tròn) và `Line` (đường thẳng), khoảng cách đại số d từ tâm đường tròn $I(a, b)$ đến đường thẳng Δ được tính toán tự động bằng phương thức tích hợp sẵn:

$$d = c.center.distance(l)$$

Dựa trên giá trị khoảng cách d so sánh trực tiếp với bán kính R (`c.radius`) của đường tròn, bài toán được phân thành ba trường hợp hình học bất biến:

1. **Trường hợp $d > R$:** Khoảng cách từ tâm đến đường thẳng lớn hơn bán kính, chứng tỏ đường thẳng nằm hoàn toàn ở phía ngoài đường tròn và hoàn toàn không có điểm chung ($\Delta \cap (C) = \emptyset$).
2. **Trường hợp $d = R$:** Khoảng cách từ tâm đến đường thẳng đúng bằng bán kính, biểu thị trạng thái đường thẳng tiếp xúc với đường tròn tại một điểm duy nhất, gọi là tiếp điểm ($|\Delta \cap (C)| = 1$).
3. **Trường hợp $d < R$:** Khoảng cách nhỏ hơn bán kính, xác định tình huống đường thẳng cắt đường tròn tại hai điểm phân biệt trên mặt phẳng tọa độ ($|\Delta \cap (C)| = 2$).

3.2 Dữ liệu đầu vào và dữ liệu ra

Dữ liệu đầu vào của chương trình gồm hai chuỗi ký tự biểu diễn phương trình đại số:

- `pt_tron_str`: Chuỗi ký tự chứa biểu thức vế trái của phương trình đường tròn (Ví dụ minh họa: `"x**2 + y**2 - 4*x - 6*y - 3"`).

- `pt_thang_str`: Chuỗi ký tự chứa biểu thức vế trái của phương trình đường thẳng (Ví dụ minh họa: `"3*x + 4*y - 12"`).

Các ràng buộc kiểm tra tính hợp lệ bắt buộc:

- $R^2 = a^2 + b^2 - c > 0$ (Đảm bảo tồn tại đường tròn thực).
- $A^2 + B^2 > 0$ (Đảm bảo phương trình biểu diễn một đường thẳng hợp lệ).

Về dữ liệu đầu ra, sau khi hoàn tất quá trình phân tích và tính toán, chương trình cung cấp các thông tin chi tiết sau:

- Tọa độ tâm $I(a, b)$ và giá trị bán kính R được trích xuất tự động từ phương trình đầu vào.
- Giá trị khoảng cách chính xác d từ tâm I đến đường thẳng Δ .
- Thông báo kết luận rõ ràng về vị trí tương đối giữa đường thẳng và đường tròn (Không giao nhau, Tiếp xúc, hoặc Cắt nhau tại hai điểm phân biệt).

3.3 Quy trình xử lý

Quy trình thực thi thuật toán được tổ chức thành một chuỗi các bước tuần tự, kiểm soát chặt chẽ từ khâu tiếp nhận dữ liệu đến khi đưa ra kết luận:

1. **Bước 1 (Khởi tạo môi trường ký hiệu):** Khai báo hai biến tọa độ không gian x, y dưới dạng đối tượng ký hiệu thông qua hàm `symbols` của thư viện SymPy.
2. **Bước 2 (Tiếp nhận và xử lý đường tròn):** Đọc chuỗi ký tự `pt_tron_str`, sử dụng `parse_expr` để phân tích thành biểu thức toán học. Trích xuất các hệ số để tính tọa độ tâm $I(a, b)$ và bình phương bán kính R^2 . Tiến hành kiểm tra điều kiện $R^2 > 0$; nếu vi phạm, chương trình in thông báo lỗi và lập tức chấm dứt thực thi (`sys.exit()`). Khởi tạo đối tượng hình học `Circle`.
3. **Bước 3 (Tiếp nhận và xử lý đường thẳng):** Đọc chuỗi ký tự `pt_thang_str`, phân tích cú pháp biểu thức và trích xuất các hệ số A, B, C . Kiểm tra điều kiện $A^2 + B^2 > 0$; nếu vi phạm, chương trình phát sinh lỗi cấu trúc và dừng thực thi. Dựa trên giá trị của hệ số B hoặc A , xác định hai điểm tọa độ hình học phân biệt p_1, p_2 thuộc đường thẳng để khởi tạo đối tượng `Line`.
4. **Bước 4 (Tính toán khoảng cách):** Gọi phương thức khoảng cách lấy từ đối tượng hình học để tính toán chính xác khoảng cách d giữa tâm đường tròn và đường thẳng.
5. **Bước 5 (Phân loại vị trí tương đối):** Thực hiện so sánh giá trị khoảng cách d với bán kính R để đưa ra kết luận cuối cùng về vị trí tương đối giữa hai đối tượng.

3.4 Thuật toán

THUẬT TOÁN: KIEM_TRA_VI_TRI_TUONG_DOI_KY_HIEU

Input: pt_tron_str, pt_thang_str

Output: Tâm I, Bán kính R, Khoảng cách d, Vị trí tương đối

BEGIN

// 1. Khởi tạo biến ký hiệu toán học

DEFINE symbols x, y

// 2. Xử lý và kiểm tra tính hợp lệ của đường tròn

READ pt_tron_str

pt_tron = PARSE_EXPR(pt_tron_str)

g = COEFFICIENT(pt_tron, x) / 2

f = COEFFICIENT(pt_tron, y) / 2

c_val = EVALUATE(pt_tron, {x: 0, y: 0})

tam_x = -g

tam_y = -f

r_binh_phuong = tam_x^2 + tam_y^2 - c_val

IF r_binh_phuong <= 0 THEN

PRINT "Lỗi: Phương trình không phải đường tròn thực!"

HALT

END IF

c = CREATE_CIRCLE(tam_x, tam_y, SQRT(r_binh_phuong))

// 3. Xử lý và kiểm tra tính hợp lệ của đường thẳng

READ pt_thang_str

pt_thang = PARSE_EXPR(pt_thang_str)

A = COEFFICIENT(pt_thang, x)

B = COEFFICIENT(pt_thang, y)

C = EVALUATE(pt_thang, {x: 0, y: 0})

IF A == 0 AND B == 0 THEN

PRINT "Lỗi: Phương trình đường thẳng không hợp lệ!"

HALT

END IF

```

IF B != 0 THEN
    p1 = POINT(0, -C/B)
    p2 = POINT(1, -(A + C)/B)
ELSE
    p1 = POINT(-C/A, 0)
    p2 = POINT(-C/A, 1)
END IF

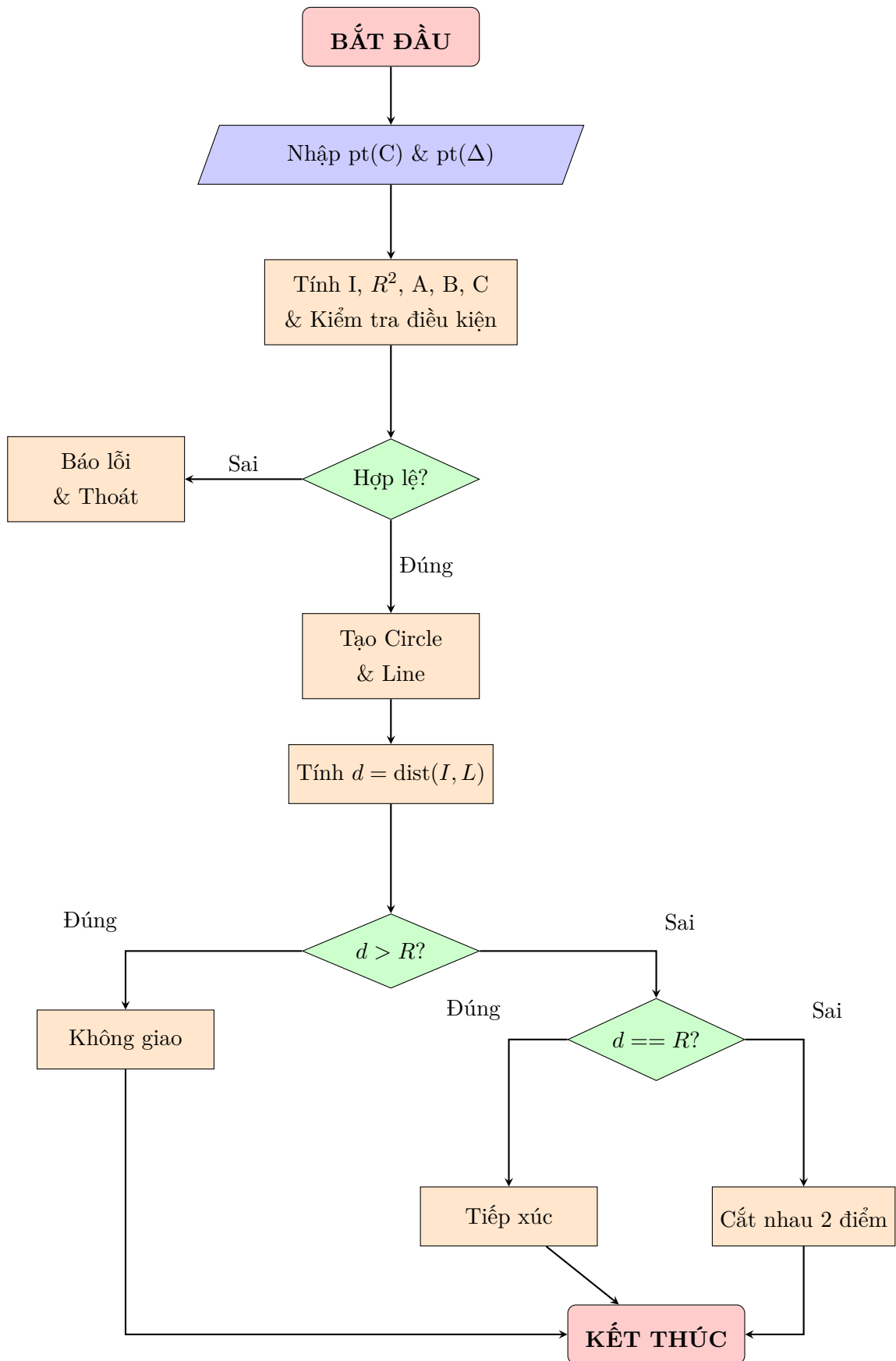
l = CREATE_LINE(p1, p2)

// 4. Tính toán khoảng cách và phân loại vị trí tương đối
dist = c.center.distance(l)

IF dist > c.radius THEN
    PRINT "Kết luận: Đường thẳng và đường tròn KHÔNG GIAO NHAU."
ELSE IF dist == c.radius THEN
    PRINT "Kết luận: Đường thẳng TIẾP XÚC với đường tròn."
ELSE
    PRINT "Kết luận: Đường thẳng CẮT đường tròn tại 2 điểm phân biệt."
END IF
END

```

3.5 Lưu đồ thuật toán



3.6 Kết luận chương 3

Chương 3 đã trình bày thành công quá trình phân tích bài toán, xây dựng mô hình đại số ký hiệu và thiết kế thuật toán chi tiết để kiểm tra vị trí tương đối giữa đường thẳng và đường tròn. Cụ thể, các kết quả đạt được trong chương bao gồm:

- **Ứng dụng mô hình ký hiệu:** Khai thác hiệu quả thư viện SymPy để tiếp nhận phương trình dưới dạng chuỗi ký tự, giúp tự động trích xuất các thông số hình học (tâm, bán kính, hệ số đường thẳng) một cách chính xác tuyệt đối mà không bị ảnh hưởng bởi sai số số thực.
- **Kiểm soát điều kiện ràng buộc:** Thiết lập cơ chế kiểm tra tính hợp lệ của phương trình đầu vào ($R^2 > 0$ đối với đường tròn và $A^2 + B^2 > 0$ đối với đường thẳng), đảm bảo tính an toàn logic trước khi thực hiện các phép tính hình học.
- **Đề xuất thuật toán và lưu đồ tối ưu:** Xây dựng hệ thống mã giả (pseudocode) và lưu đồ thuật toán mạch lạc, ngắn gọn, minh họa trực quan quy trình từ khâu xử lý dữ liệu đầu vào đến bước phân loại ba trường hợp vị trí tương đối (không giao nhau, tiếp xúc, cắt nhau tại hai điểm).

Nội dung của chương này tạo nền tảng lý thuyết và thuật toán vững chắc, làm cơ sở để tiến hành hiện thực hóa mã nguồn chương trình và kiểm thử thực nghiệm ở chương tiếp theo.

4 XÂY DỰNG CHƯƠNG TRÌNH VÀ PHÂN TÍCH MÃ NGUỒN

Trong chương này, nhóm tiến hành triển khai các khái niệm hình học giải tích thành chương trình Python hoàn chỉnh. Đoạn mã nguồn được cung cấp đóng vai trò là một "hệ thống tính toán ký hiệu tương tác" (Interactive Symbolic Computation System), cho phép người dùng nhập trực tiếp biểu thức phương trình dưới dạng văn bản tự nhiên, sau đó hệ thống tự động phân tích cú pháp, trích xuất tham số, thiết lập các đối tượng hình học và đưa ra kết luận chính xác.

4.1 Thiết lập môi trường và phân tích biểu thức người dùng (parse_expr)

```
import sys
from sympy import symbols, Eq
from sympy.parsing.sympy_parser import parse_expr
from sympy.geometry import Point, Line, Circle

# Khoi tao hai bien ky hieu x va y
x, y = symbols('x y')
```

- Phân tích kỹ thuật:
 - Chương trình khởi tạo hai biến toán học ký hiệu x và y bằng hàm `symbols('x y')`. Điều này thông báo cho SymPy biết rằng x và y không phải là các biến số thông thường trong lập trình mà là các ẩn số đại số.
 - Sử dụng hàm `parse_expr` từ mô-đun `sympy.parsing.sympy_parser` giúp nâng cao tính linh hoạt của chương trình. Thay vì yêu cầu người dùng phải nhập rời rạc từng hệ số A, B, C, a, b, R , người dùng có thể nhập thẳng biểu thức dạng đại số quen thuộc (ví dụ: $x^2 + y^2 - 4x - 6y - 3$). `parse_expr` sẽ dịch chuỗi văn bản này thành cấu trúc cây biểu thức toán học (AST) hiểu được bởi SymPy.

4.2 Tự động trích xuất tham số và kiểm tra tính hợp lệ của đường tròn

Phần đầu của khối lệnh `try` chịu trách nhiệm xử lý phương trình đường tròn:

```
pt_tron_str = input("Nhap ve trai PT duong tron (ep ve = 0): ")
pt_tron = parse_expr(pt_tron_str)

# Dua ve dang tong quat: x^2 + y^2 + 2gx + 2fy + c = 0
g = pt_tron.coeff(x) / 2
```

```

f = pt_tron.coeff(y) / 2
c_val = pt_tron.subs({x: 0, y: 0})

tam_x = -g
tam_y = -f
r_binh_phuong = tam_x**2 + tam_y**2 - c_val

if r_binh_phuong <= 0:
    print("Loi: Phương trình không phải là một đường tròn thực!")
    sys.exit()

c = Circle(Point(tam_x, tam_y), r_binh_phuong**0.5)

```

- Phân tích thuật toán toán học:

- **Trích xuất hệ số (coeff):** Phương pháp `pt_tron.coeff(x)` và `pt_tron.coeff(y)` tự động tìm hệ số của bậc nhất ứng với biến x và y trong biểu thức tổng quát dạng khai triển:

$$x^2 + y^2 + 2gx + 2fy + c = 0$$

Từ đó, ta suy ra hệ số dịch tâm $g = \frac{\text{coeff}(x)}{2}$ và $f = \frac{\text{coeff}(y)}{2}$.

- **Xác định hằng số tự do (c):** Bằng cách thay giá trị $x = 0, y = 0$ vào biểu thức thông qua phương thức `.subs({x: 0, y: 0})`, chương trình thu được hằng số tự do c .
- **Tính tọa độ tâm và bán kính:** Tọa độ tâm đường tròn $I(x_0, y_0)$ được xác định qua công thức $-g$ và $-f$. Bình phương bán kính được tính theo công thức:

$$R^2 = x_0^2 + y_0^2 - c$$

- **Kiểm tra điều kiện tồn tại:** Nếu $R^2 \leq 0$, biểu thức không thỏa mãn định nghĩa đường tròn thực trong mặt phẳng Euclid, chương trình lập tức báo lỗi và dừng an toàn bằng `sys.exit()`.
- **Khởi tạo đối tượng Circle:** Khi $R^2 > 0$, đối tượng hình học `Circle` của SymPy được khởi tạo với tâm là một `Point(tam_x, tam_y)` và bán kính $R = \sqrt{R^2}$.

4.3 Xử lý và chuyển đổi phương trình đường thẳng thành đối tượng Line

Phần tiếp theo của chương trình xử lý phương trình đường thẳng dạng tổng quát $Ax + By + C = 0$:

```

pt_thang_str = input("Nhập về trai PT duong thang (ep ve = 0): ")
pt_thang = parse_expr(pt_thang_str)

```

```

A = pt_thang.coeff(x)
B = pt_thang.coeff(y)
C = pt_thang.subs({x: 0, y: 0})

if A == 0 and B == 0:
    print("Loi: Phuong trinh duong thang khong hop le!")
    sys.exit()

if B != 0:
    p1 = Point(0, -C/B)
    p2 = Point(1, -(A + C)/B)
else:
    p1 = Point(-C/A, 0)
    p2 = Point(-C/A, 1)

l = Line(p1, p2)

```

- **Phân tích kỹ thuật lập trình:**

- **Trích xuất hệ số đường thẳng:** Tương tự như đường tròn, chương trình trích xuất các hệ số A (hệ số của x), B (hệ số của y) và hằng số tự do C thông qua việc lấy giá trị tại gốc tọa độ.
- **Kiểm tra tính hợp lệ:** Điều kiện $A = 0$ và $B = 0$ đồng thời xảy ra sẽ khiến phương trình vô nghĩa ($0 = C$), chương trình sẽ chặn lại để tránh lỗi toán học.
- **Kỹ thuật sinh 2 điểm thuộc đường thẳng (Line instantiation):**
 - * Thư viện hình học của SymPy (`sympy.geometry.Line`) yêu cầu khởi tạo thông qua các đối tượng điểm (`Point`).
 - * Chương trình sử dụng cấu trúc nhánh `if-else` để xử lý:
 - Trường hợp $B \neq 0$ (Đường thẳng không song song với trục tung): Cho $x = 0 \implies y = -C/B$ (điểm p_1); cho $x = 1 \implies y = -(A + C)/B$ (điểm p_2).
 - Trường hợp $B = 0$ (Đường thẳng song song hoặc trùng với trục tung, dạng $Ax + C = 0 \implies x = -C/A$): Cho $y = 0 \implies x = -C/A$ (điểm p_1); cho $y = 1 \implies x = -C/A$ (điểm p_2).
 - * Nhờ cách này, mọi đường thẳng trong mặt phẳng đều được chuyển đổi thành công thành đối tượng `Line` của thư viện SymPy.

4.4 Tính khoảng cách và kiểm tra vị trí tương đối

Khối lệnh cuối cùng thực hiện việc tính toán hình học cốt lõi và đưa ra kết luận.

```
# 3. Tính toán vị trí tương đối
dist = c.center.distance(l)
print(f"\nKhoang cach tu tam den duong thang d = {dist}")

if dist > c.radius:
    print("=> KET LUAN: Duong thang va duong tron KHONG GIAO NHAU
        .")
elif dist == c.radius:
    print("=> KET LUAN: Duong thang TIEP XUC voi duong tron.")
else:
    print("=> KET LUAN: Duong thang CAT duong tron tai 2 diem
        phan biet.")
```

- **Phân tích tính ưu việt của SymPy trong tính toán:**

- Phương thức `c.center.distance(l)` tự động gọi thuật toán hình học giải tích để tính khoảng cách từ tâm đường tròn (kiểu `Point`) đến đường thẳng `l` (kiểu `Line`).
- Giá trị `dist` trả về dưới dạng **ký hiệu với độ chính xác cao** (chứa căn thức, phân số tối giản nếu các hệ số đầu vào là số nguyên/hữu tỉ), hoàn toàn không bị ảnh hưởng bởi sai số dấu phẩy động (floating-point error) như khi dùng NumPy hay thư viện số học thông thường.
- Cấu trúc rẽ nhánh `if-elif-else` so sánh trực tiếp khoảng cách `dist` với bán kính `c.radius` để phân loại chính xác 3 trạng thái hình học kinh điển một cách minh bạch và gọn gàng.

4.5 Cơ chế kiểm soát ngoại lệ (try-except)

```
except Exception as e:
    print(f"Da xay ra loi nhap lieu hoac cu phap: {e}")
```

- Khối lệnh này đảm bảo chương trình không bị crash (đột ngột dừng toàn bộ tiến trình) khi người dùng nhập sai cú pháp biểu thức (ví dụ thiếu dấu nhân `*`, gõ sai tên biến, hoặc nhập các ký tự không hợp lệ). Mọi ngoại lệ được bắt lại và thông báo trực quan cho người dùng, giúp tăng tính thân thiện và ổn định cho hệ thống phần mềm.

5 THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Các trường hợp thử nghiệm

Chương trình được xây dựng để xác định vị trí tương đối giữa đường thẳng và đường tròn trên mặt phẳng tọa độ Oxy . Để kiểm chứng đầy đủ chức năng, tính chính xác và độ bền của chương trình, chúng ta thiết kế 4 nhóm trường hợp thử nghiệm bao quát toàn bộ các tình huống có thể xảy ra:

STT	Loại trường hợp	Mục đích kiểm tra
1	Đường thẳng cắt đường tròn tại hai điểm phân biệt	Xác minh chương trình tính tâm, bán kính, khoảng cách chính xác và nhận dạng đúng trạng thái cắt nhau
2	Đường thẳng tiếp xúc với đường tròn	Kiểm tra việc nhận diện điều kiện đặc biệt $d = R$ không nhầm lẫn với hai trường hợp còn lại
3	Đường thẳng không giao nhau	Xác nhận chương trình phân biệt rõ trường hợp $d > R$
4	Dữ liệu không hợp lệ	Đánh giá khả năng xử lý lỗi: thông báo rõ ràng, chương trình không bị treo hoặc dừng đột ngột

Bảng 6.1: Các nhóm trường hợp thử nghiệm

1. Trường hợp 1: Đường thẳng cắt đường tròn

Điều kiện: Khoảng cách từ tâm đường tròn đến đường thẳng nhỏ hơn bán kính ($d < R$).

- Phương trình đường tròn: $x^2 + y^2 - 4x - 6y - 3 = 0$
- Phương trình đường thẳng: $3x + 4y - 12 = 0$
- Tính toán tham chiếu:**
 - Tâm $I(2; 3)$; Bán kính $R = 4$
 - Khoảng cách: $d = \frac{|3 \cdot 2 + 4 \cdot 3 - 12|}{\sqrt{3^2 + 4^2}} = \frac{6}{5} = 1.2$
- So sánh:** $1.2 < 4 \rightarrow$ Đường thẳng cắt đường tròn tại hai điểm phân biệt.

2. Trường hợp 2: Đường thẳng tiếp xúc với đường tròn

Điều kiện: Khoảng cách từ tâm đến đường thẳng bằng bán kính ($d = R$).

- Phương trình đường tròn: $x^2 + y^2 - 25 = 0$

- Phương trình đường thẳng: $3x + 4y - 25 = 0$
- **Tính toán tham chiếu:**
 - Tâm $I(0; 0)$; Bán kính $R = 5$
 - Khoảng cách: $d = \frac{|3 \cdot 0 + 4 \cdot 0 - 25|}{\sqrt{3^2 + 4^2}} = \frac{25}{5} = 5$
- **So sánh:** $d = R = 5 \rightarrow$ Đường thẳng tiếp xúc với đường tròn.

3. Trường hợp 3: Đường thẳng không giao nhau

Điều kiện: Khoảng cách từ tâm đến đường thẳng lớn hơn bán kính ($d > R$).

- Phương trình đường tròn: $x^2 + y^2 - 4x + 2y - 4 = 0$
- Phương trình đường thẳng: $4x - 3y + 24 = 0$
- **Tính toán tham chiếu:**
 - Tâm $I(2; -1)$; Bán kính $R = 3$
 - Khoảng cách: $d = \frac{|4 \cdot 2 - 3 \cdot (-1) + 24|}{\sqrt{4^2 + (-3)^2}} = \frac{35}{5} = 7$
- **So sánh:** $7 > 3 \rightarrow$ Đường thẳng và đường tròn không giao nhau.

4. Trường hợp 4: Dữ liệu không hợp lệ

Kiểm tra các tình huống nhập liệu không thỏa mãn điều kiện tồn tại hình:

- Phương trình không phải đường tròn thực: $x^2 + y^2 - 2x - 4y + 5 = 0$ (vì $R^2 = 0$)
- Phương trình đường tròn không tồn tại trên mặt phẳng thực: $x^2 + y^2 + 4 = 0$ (vì $R^2 < 0$)
- Phương trình đường thẳng không xác định: $0x + 0y + 7 = 0$
- Lỗi cú pháp khi nhập biểu thức: `x**2+y**2-3*x+` (thiếu hạng tử cuối)

5.2 Kết quả

Chạy chương trình với từng bộ số liệu đã thiết kế, kết quả thu được chi tiết như sau:

1. Kết quả trường hợp 1 – Đường thẳng cắt đường tròn

Dữ liệu nhập vào chương trình:

Nhập vế trái PT đường tròn (ép về = 0): `x**2 + y**2 - 4*x - 6*y - 3`
 Nhập vế trái PT đường thẳng (ép về = 0): `3*x + 4*y - 12`

Kết quả chương trình trả về:

KIỂM TRA VỊ TRÍ TƯƠNG ĐỐI TỰ ĐỘNG

Ví dụ đường tròn: $x^2 + y^2 - 4x - 6y - 3$

Ví dụ đường thẳng: $3x + 4y - 12$

-> Khớp dữ liệu thành công! Tâm $I(2, 3)$, Bán kính $R=4.0$

Khoảng cách từ tâm đến đường thẳng $d=6/5=1.2$

=> KẾT LUẬN: Đường thẳng CẮT đường tròn tại 2 điểm phân biệt.

Đối chiếu: Tâm, bán kính, khoảng cách và kết luận hoàn toàn trùng khớp với tính toán tham chiếu. ✓

2. Kết quả trường hợp 2 – Đường thẳng tiếp xúc

Dữ liệu nhập vào chương trình:

Nhập vế trái PT đường tròn: $x^2 + y^2 - 25$

Nhập vế trái PT đường thẳng: $3x + 4y - 25$

Kết quả chương trình trả về:

-> Khớp dữ liệu thành công! Tâm $I(0, 0)$, Bán kính $R=5.0$

Khoảng cách từ tâm đến đường thẳng $d=5.0$

=> KẾT LUẬN: Đường thẳng TIẾP XÚC với đường tròn.

Đối chiếu: Kết quả khớp với lý thuyết, chương trình nhận diện chính xác điều kiện tiếp xúc. ✓

3. Kết quả trường hợp 3 – Đường thẳng không giao nhau

Dữ liệu nhập vào chương trình:

Nhập vế trái PT đường tròn: $x^2 + y^2 - 4x + 2y - 4$

Nhập vế trái PT đường thẳng: $4x - 3y + 24$

Kết quả chương trình trả về:

-> Khớp dữ liệu thành công! Tâm $I(2, -1)$, Bán kính $R=3.0$

Khoảng cách từ tâm đến đường thẳng $d=7.0$

=> KẾT LUẬN: Đường thẳng và đường tròn KHÔNG GIAO NHAU.

Đối chiếu: Hoàn toàn chính xác, $d = 7 > R = 3$ như tính toán thủ công. ✓

4. Kết quả trường hợp 4 – Xử lý dữ liệu không hợp lệ

Tất cả các tình huống lỗi đều được chương trình xử lý linh hoạt, thông báo rõ ràng, chương trình kết thúc có kiểm soát, không bị treo hoặc hỏng.

Dữ liệu nhập	Kết quả chương trình	Đánh giá
$x^{**2} + y^{**2} - 2*x - 4*y + 5$	Lỗi: Phương trình không phải là một đường tròn thực!	Đúng
$x^{**2} + y^{**2} + 4$	Lỗi: Phương trình không phải là một đường tròn thực!	Đúng
$0*x + 0*y + 7$	Lỗi: Phương trình đường thẳng không hợp lệ!	Đúng
$x^{**2} + y^{**2} - 3*x +$	Đã xảy ra lỗi nhập liệu hoặc cú pháp:	Đúng

Bảng 6.2: Kết quả xử lý các trường hợp dữ liệu không hợp lệ

5.3 Đánh giá chương trình

Dựa trên kết quả thực nghiệm thu được, tiến hành đánh giá chi tiết về các mặt chất lượng chương trình như sau:

1. Độ chính xác

- Các kết quả tính toán tâm, bán kính và khoảng cách đều trùng khớp với kết quả tính toán thủ công theo công thức hình học giải tích.
- Việc xác định trạng thái vị trí tương đối giữa đường thẳng và đường tròn phù hợp với từng trường hợp đã thiết kế.
- Sử dụng thư viện SymPy xử lý biểu thức dưới dạng ký hiệu nên tránh được sai số làm tròn ở các bước tính trung gian, đảm bảo kết quả chính xác.
- Việc khởi tạo đối tượng đường thẳng thông qua hai điểm thuộc đường thẳng giúp hạn chế lỗi do hệ số bằng không trong phương trình tổng quát.

2. Tính ổn định và khả năng xử lý lỗi

- Toàn bộ luồng thực thi chính được bao bọc trong khối `try...except`, giúp phát hiện và xử lý kịp thời các lỗi về cú pháp, kiểu dữ liệu hoặc lỗi tính toán mà không làm chương trình dừng đột ngột.
- Có các điều kiện kiểm tra sớm: phát hiện khi bán kính bình phương không dương, khi cả hai hệ số của x và y trong phương trình đường thẳng đều bằng không. Nhờ đó ngăn chặn được các lỗi logic trước khi thực hiện các phép tính tiếp theo.
- Thông báo lỗi bằng ngôn ngữ tiếng Việt rõ ràng, dễ hiểu, giúp người dùng xác định và sửa lỗi nhập liệu một cách thuận lợi.

3. Tính hữu dụng và dễ sử dụng

- Giao diện dòng lệnh đơn giản, ngay đầu chương trình đã hiển thị ví dụ cụ thể giúp người dùng dễ dàng tham khảo và nhập đúng định dạng.
- Người dùng không cần thực hiện bất kỳ phép biến đổi trung gian nào, chỉ cần sao chép vế trái của phương trình đã cho và nhập vào chương trình.
- Kết quả trả về có cấu trúc rõ ràng: thông tin tâm, bán kính, giá trị khoảng cách và cuối cùng là kết luận trực tiếp.
- Chương trình có tính tổng quát, hoạt động với mọi phương trình đường tròn viết dưới dạng tổng quát chuẩn và mọi phương trình đường thẳng, không bị giới hạn trong một bộ số liệu cụ thể.

4. Hạn chế tồn tại

Bên cạnh những kết quả đạt được, chương trình vẫn còn một số điểm cần hoàn thiện:

- Chưa kiểm tra và chuẩn hóa hệ số của x^2 và y^2 : nếu người dùng nhập phương trình có hệ số bậc hai khác 1 (ví dụ: $2x^2 + 2y^2 - 4x + \dots$), chương trình sẽ xử lý chưa chính xác.
- Chưa tính và hiển thị tọa độ các điểm giao hoặc điểm tiếp xúc: chỉ thông báo trạng thái tương đối nhưng chưa đưa ra tọa độ cụ thể.
- Việc so sánh khoảng cách và bán kính sử dụng toán tử `==` có thể gặp khó khăn trong các trường hợp giá trị tính ra là số thập phân gần bằng do sai số làm tròn.
- Chưa có hình ảnh minh họa trực quan, người dùng chỉ theo dõi được bằng số liệu.

5. Hướng phát triển

Từ những phân tích trên, có thể nâng cấp chương trình theo các hướng sau:

- Thêm bước kiểm tra và chuẩn hóa hệ số của x^2 và y^2 về bằng 1 trước khi tính toán tâm và bán kính.
- Tích hợp giải hệ phương trình để xác định tọa độ chính xác các điểm giao hoặc điểm tiếp xúc và hiển thị ra màn hình.
- Thay đổi cách so sánh: sử dụng một khoảng sai số nhỏ chấp nhận được ($\epsilon \approx 10^{-9}$) thay vì so sánh bằng trực tiếp để tránh nhầm lẫn do sai số.
- Tích hợp thư viện vẽ đồ thị để hiển thị trực quan đường tròn, đường thẳng, tâm và các điểm đặc biệt trên mặt phẳng tọa độ.

6. Kết luận chương

Chương trình sau khi xây dựng và kiểm thử thực tế đã chứng minh đáp ứng đầy đủ yêu cầu đề ra: xác định chính xác vị trí tương đối giữa đường thẳng và đường tròn trong các trường hợp cơ bản, xử lý tốt các tình huống lỗi phổ biến, giao diện thân thiện, dễ sử dụng và dễ mở rộng. Chương trình có thể được ứng dụng làm công cụ hỗ trợ học tập và giảng dạy môn Hình học giải tích, đồng thời là nền tảng tốt để phát triển các chương trình hình học phức tạp hơn trong tương lai.

6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Phân tích chi tiết mục này nhằm tổng kết toàn diện giá trị học thuật và thực tiễn của đề tài "Sử dụng SymPy tự động kiểm tra vị trí tương đối giữa đường thẳng và đường tròn", đồng thời chỉ ra những giới hạn hiện tại và định hướng mở rộng trong tương lai.

6.1 Kết quả đạt được

Đề tài đã hoàn thành đầy đủ các mục tiêu nghiên cứu ban đầu đặt ra, thể hiện qua các khía cạnh cụ thể sau:

- **Hệ thống hóa nền tảng toán học:** Xây dựng thành công cơ sở lý thuyết chặt chẽ về hình học giải tích liên quan đến phương trình đường thẳng, đường tròn, khoảng cách từ điểm đến đường thẳng và phương pháp biện luận vị trí tương đối.
- **Khai thác hiệu quả công nghệ Python & SymPy:** Ứng dụng hiệu quả thư viện tính toán ký hiệu SymPy vào việc xử lý các biểu thức đại số, tự động phân tích cú pháp chuỗi đầu vào (`parse_expr`), trích xuất linh hoạt thông số hình học (tâm, bán kính, hệ số đường thẳng) và tính toán khoảng cách với độ chính xác cao mà không bị ảnh hưởng bởi sai số làm tròn số thực.
- **Xây dựng chương trình tự động hóa:** Hoàn thiện mã nguồn chương trình tương tác trực tiếp với người dùng, có cơ chế kiểm tra ngoại lệ (`try...except`) và kiểm soát dữ liệu đầu vào chặt chẽ (phát hiện đường tròn không thực, đường thẳng vô nghiệm hoặc hệ số không hợp lệ), đảm bảo tính ổn định cao trong quá trình thực thi.

6.2 Hạn chế

Mặc dù chương trình đã giải quyết trọn vẹn bài toán đặt ra trong phạm vi không gian 2 chiều, đề tài vẫn tồn tại một số điểm hạn chế nhất định:

- **Phạm vi hình học còn hẹp:** Hệ thống hiện chỉ tập trung xử lý trên mặt phẳng tọa độ hai chiều (Oxy) đối với các đối tượng cơ bản là đường thẳng và đường tròn, chưa mở rộng sang các đường conic khác (như Elip, Hypebol, Parabol) hoặc không gian 3 chiều ($Oxyz$).
- **Giao diện người dùng dạng dòng lệnh (CLI):** Chương trình hiện chạy hoàn toàn qua môi trường cửa sổ lệnh văn bản, chưa tích hợp giao diện đồ họa trực quan (GUI) hay giao diện web để người dùng phổ thông dễ dàng tương tác và quan sát hình ảnh minh họa trực tiếp.
- **Hiệu năng tính toán ký hiệu:** Việc sử dụng tính toán ký hiệu giúp đạt độ chính xác tuyệt đối nhưng có thể tiêu tốn nhiều tài nguyên xử lý hơn so với thư viện tính toán số học thuần túy (NumPy) khi áp dụng vào các bài toán có quy mô lớn hoặc hệ phương trình phức tạp.

6.3 Hướng phát triển

Để nâng cấp và hoàn thiện đề tài trong tương lai, một số hướng phát triển tiềm năng có thể được triển khai:

- **Mở rộng không gian và đối tượng hình học:** Phát triển mã nguồn để giải quyết các bài toán phức tạp hơn như vị trí tương đối giữa mặt phẳng và mặt cầu trong không gian 3 chiều (*Oxyz*), hoặc xét giao cắt giữa đường thẳng và các đường conic tổng quát.
- **Phát triển giao diện đồ họa (GUI / Web App):** Xây dựng ứng dụng trực quan bằng các thư viện giao diện như PyQt, Tkinter hoặc đưa lên nền tảng Web sử dụng Streamlit / Flask kết hợp với `matplotlib` để người dùng vừa nhập phương trình, vừa vẽ trực quan hình học ngay trên màn hình.
- **Tích hợp giải và hiển thị bước giải chi tiết:** Nâng cấp chương trình không chỉ đưa ra kết quả cuối cùng mà còn tự động in ra các bước giải chi tiết (như cách lập phương trình hoành độ giao điểm, tính toán delta) nhằm hỗ trợ đắc lực cho công tác giảng dạy và học tập môn Toán hình học.

Tài liệu

- [1] Bộ Giáo dục và Đào tạo. (2024). *Bài 16: Vị trí tương đối của đường thẳng và đường tròn*. Trong *Toán 9* (Tập 1) (tr. 99–103). Nhà xuất bản Giáo dục Việt Nam.
- [2] Đoàn Quỳnh. (2009). *Giáo trình Đại số tuyến tính và Hình học giải tích*. Nhà xuất bản Đại học Quốc gia.
- [3] Trần Hạnh Nhi, & Dương Anh Đức. (2000). *Nhập môn cấu trúc dữ liệu và thuật toán*. Khoa Công nghệ thông tin, Đại học Khoa học Tự nhiên TP.HCM.
- [4] Nguyễn Ngô Bảo Trân. (2005). *Giáo trình cấu trúc dữ liệu và giải thuật*. Trường Đại học Bách Khoa TP.HCM.
- [5] SymPy Development Team. (2025). *Assumptions*. SymPy Documentation.
- [6] OpenAI. (2026). *ChatGPT* (Phiên bản ChatGPT-4o) [Large language model].
- [7] Google. (2026). *Gemini* (Phiên bản Gemini 1.5 Pro) [Large language model].